

Register File UVM Verification

Generated by Doxygen 1.16.1

1 Topic Index	1
1.1 Topics	1
2 Directory Hierarchy	3
2.1 Directories	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Topic Documentation	11
6.1 Modules	11
6.2 Testbench components	11
6.2.1 Detailed Description	11
6.3 Interfaces	11
6.3.1 Detailed Description	12
6.3.2 Function Documentation	12
6.3.2.1 s_interface()	12
6.4 UVM verification components	12
6.4.1 Detailed Description	12
7 Directory Documentation	13
7.1 filter Directory Reference	13
7.1.1 Detailed Description	13
7.1.2 SystemVerilog Doxygen filter	13
7.2 reference_model Directory Reference	13
7.3 rtl Directory Reference	13
7.4 tb Directory Reference	14
7.5 uvm Directory Reference	14
8 Class Documentation	15
8.1 agent Class Reference	15
8.1.1 Detailed Description	16
8.1.2 Member Function Documentation	16
8.1.2.1 build_phase()	16
8.1.2.2 connect_phase()	16
8.1.2.3 new()	16
8.1.2.4 uvm_component_utils()	17
8.1.3 Member Data Documentation	17
8.1.3.1 m_monitor	17

8.1.3.2 m_sequencer	17
8.2 coverage Class Reference	17
8.2.1 Detailed Description	18
8.2.2 Member Function Documentation	18
8.2.2.1 cg_register_file()	18
8.2.2.2 new()	19
8.2.2.3 uvm_component_utils()	19
8.2.2.4 write()	19
8.3 driver Class Reference	19
8.3.1 Detailed Description	20
8.3.2 Member Function Documentation	21
8.3.2.1 build_phase()	21
8.3.2.2 drive_item()	21
8.3.2.3 new()	21
8.3.2.4 run_phase()	21
8.3.2.5 uvm_component_utils()	21
8.3.3 Member Data Documentation	21
8.3.3.1 req_pkt	21
8.4 env Class Reference	22
8.4.1 Detailed Description	23
8.4.2 Member Function Documentation	23
8.4.2.1 build_phase()	23
8.4.2.2 connect_phase()	23
8.4.2.3 new()	23
8.4.2.4 uvm_component_utils()	23
8.4.3 Member Data Documentation	23
8.4.3.1 m_coverage	23
8.4.3.2 m_scoreboard	24
8.5 monitor Class Reference	24
8.5.1 Detailed Description	25
8.5.2 Member Function Documentation	25
8.5.2.1 build_phase()	25
8.5.2.2 new()	25
8.5.2.3 run_phase()	25
8.5.2.4 uvm_component_utils()	25
8.5.3 Member Data Documentation	26
8.5.3.1 mon_analysis_port	26
8.6 pkt Class Reference	26
8.6.1 Detailed Description	27
8.6.2 Member Function Documentation	27
8.6.2.1 uvm_field_int() [1/2]	27
8.6.2.2 uvm_field_int() [2/2]	27

8.6.2.3 uvm_object_utils_begin()	27
8.6.3 Member Data Documentation	28
8.6.3.1 i_data_WB	28
8.6.3.2 i_instr_ID	28
8.6.3.3 i_rd_WB	28
8.6.3.4 i_write_en_WB	28
8.6.3.5 o_rs1_ID	28
8.6.3.6 o_rs2_ID	28
8.7 register_file_sequence Class Reference	29
8.7.1 Detailed Description	30
8.7.2 Member Function Documentation	30
8.7.2.1 body()	30
8.7.2.2 new()	30
8.7.2.3 uvm_object_utils()	30
8.7.3 Member Data Documentation	30
8.7.3.1 num_packets	30
8.8 s_interface Interface Reference	30
8.8.1 Detailed Description	31
8.9 scoreboard Class Reference	31
8.9.1 Detailed Description	32
8.9.2 Member Function Documentation	32
8.9.2.1 build_phase()	32
8.9.2.2 check_phase()	32
8.9.2.3 new()	32
8.9.2.4 uvm_component_utils()	32
8.9.2.5 write()	32
8.9.3 Member Data Documentation	33
8.9.3.1 ap_imp	33
8.9.3.2 num_errors	33
8.10 sequencer Class Reference	33
8.10.1 Detailed Description	34
8.10.2 Member Function Documentation	34
8.10.2.1 uvm_component_utils()	34
8.10.3 Member Data Documentation	34
8.10.3.1 parent	34
8.11 test Class Reference	35
8.11.1 Detailed Description	36
8.11.2 Member Function Documentation	36
8.11.2.1 build_phase()	36
8.11.2.2 end_of_elaboration_phase()	36
8.11.2.3 new()	36
8.11.2.4 run_phase()	36

8.11.2.5 uvm_component_utils()	36
8.11.3 Member Data Documentation	36
8.11.3.1 m_sequence	36
9 File Documentation	37
9.1 filter/README.md File Reference	37
9.2 reference_model/reference_model.c File Reference	37
9.2.1 Detailed Description	38
9.2.2 Macro Definition Documentation	38
9.2.2.1 NUM_REGS	38
9.2.3 Function Documentation	38
9.2.3.1 register_file_golden()	38
9.3 rtl/register_file.v File Reference	38
9.3.1 Function Documentation	39
9.3.1.1 always()	39
9.3.2 Variable Documentation	39
9.3.2.1 clk	39
9.3.2.2 i_data_WB	39
9.3.2.3 i_instr_ID	39
9.3.2.4 i_rd_WB	39
9.3.2.5 i_rst_ID	39
9.3.2.6 i_write_en_WB	39
9.3.2.7 o_rs1_ID	40
9.3.2.8 o_rs2_ID	40
9.3.2.9 registers	40
9.4 rtl/register_file_wrapper.sv File Reference	40
9.4.1 Detailed Description	40
9.5 tb/s_interface.sv File Reference	40
9.5.1 Detailed Description	40
9.6 tb/tb_top.sv File Reference	40
9.6.1 Detailed Description	41
9.6.2 Function Documentation	41
9.6.2.1 run_test()	41
9.6.2.2 U_REGISTER_FILE_WRAPPER()	41
9.6.2.3 vif()	41
9.6.3 Variable Documentation	41
9.6.3.1 forever	41
9.6.3.2 rst	42
9.7 uvm/agent.sv File Reference	42
9.7.1 Detailed Description	42
9.8 agent.sv	42
9.9 uvm/coverage.sv File Reference	43

9.9.1 Detailed Description	43
9.10 coverage.sv	44
9.11 uvm/driver.sv File Reference	44
9.11.1 Detailed Description	45
9.12 driver.sv	45
9.13 uvm/env.sv File Reference	45
9.13.1 Detailed Description	46
9.14 env.sv	46
9.15 uvm/monitor.sv File Reference	47
9.15.1 Detailed Description	47
9.16 monitor.sv	47
9.17 uvm/pkt.sv File Reference	48
9.17.1 Detailed Description	48
9.18 pkt.sv	49
9.19 uvm/register_file_pkg.sv File Reference	49
9.19.1 Detailed Description	49
9.20 uvm/register_file_sequence.sv File Reference	50
9.20.1 Detailed Description	50
9.21 register_file_sequence.sv	50
9.22 uvm/scoreboard.sv File Reference	51
9.22.1 Detailed Description	51
9.22.2 Function Documentation	51
9.22.2.1 register_file_golden()	51
9.23 scoreboard.sv	52
9.24 uvm/sequencer.sv File Reference	53
9.24.1 Detailed Description	53
9.25 sequencer.sv	53
9.26 uvm/test.sv File Reference	54
9.26.1 Detailed Description	54
9.27 test.sv	54
Index	55

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

Modules	11
Testbench components	11
Interfaces	11
UVM verification components	12

Chapter 2

Directory Hierarchy

2.1 Directories

filter	13
reference_model	13
reference_model.c	37
rtl	13
register_file.v	38
register_file_wrapper.sv	40
tb	14
s_interface.sv	40
tb_top.sv	40
uvm	14
agent.sv	42
coverage.sv	43
driver.sv	44
env.sv	45
monitor.sv	47
pkt.sv	48
register_file_pkg.sv	49
register_file_sequence.sv	50
scoreboard.sv	51
sequencer.sv	53
test.sv	54

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

s_interface	30
uvm_agent	
agent	15
uvm_driver	
driver	19
uvm_env	
env	22
uvm_monitor	
monitor	24
uvm_scoreboard	
scoreboard	31
uvm_sequence	
register_file_sequence	29
uvm_sequence_item	
pkt	26
uvm_sequencer	
sequencer	33
uvm_subscriber	
coverage	17
uvm_test	
test	35

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

agent	UVM agent that connects the sequencer, driver, and monitor	15
coverage	UVM coverage subscriber for register-file transaction sampling	17
driver	UVM driver for register-file transactions	19
env	Top-level UVM environment for the register-file verification bench	22
monitor	UVM monitor for register-file transaction observation	24
pkt	Sequence item that models one register-file access	26
register_file_sequence	UVM sequence that drives register-file transactions until coverage reaches the target	29
s_interface	Register-file stimulus/response interface shared by the DUT and the UVM environment	30
scoreboard	UVM scoreboard for register-file functional checking	31
sequencer	UVM sequencer for register-file transactions	33
test	Top-level UVM test for the register-file verification bench	35

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

reference_model/ reference_model.c	
Golden reference model for the register file, called from the UVM scoreboard via DPI-C	37
rtl/ register_file.v	38
rtl/ register_file_wrapper.sv	
Wrapper that connects the register-file RTL to the UVM interface	40
tb/ s_interface.sv	
Register-file stimulus/response interface shared by the DUT and the UVM environment	40
tb/ tb_top.sv	
Top-level testbench that instantiates the DUT, interface, and UVM test	40
uvm/ agent.sv	
UVM agent that bundles the sequencer, driver, and monitor	42
uvm/ coverage.sv	
Coverage collector for register-file transactions	43
uvm/ driver.sv	
UVM driver that translates register-file transactions into interface signal activity	44
uvm/ env.sv	
UVM environment that instantiates the agent, scoreboard, and coverage collector	45
uvm/ monitor.sv	
UVM monitor that samples register-file transactions and publishes them to subscribers	47
uvm/ pkt.sv	
Transaction item used to model one register-file access in the UVM environment	48
uvm/ register_file_pkg.sv	
Package containing the register-file UVM verification components	49
uvm/ register_file_sequence.sv	
UVM sequence that drives register-file transactions until coverage reaches the target	50
uvm/ scoreboard.sv	
Scoreboard that checks register-file transactions against the golden reference model	51
uvm/ sequencer.sv	
UVM sequencer that provides transactions to the driver	53
uvm/ test.sv	
Top-level UVM test that builds the environment and starts the main sequence	54

Chapter 6

Topic Documentation

6.1 Modules

6.2 Testbench components

Interface and top-level testbench modules used by the verification environment.

Files

- file [s_interface.sv](#)
Register-file stimulus/response interface shared by the DUT and the UVM environment.
- file [tb_top.sv](#)
Top-level testbench that instantiates the DUT, interface, and UVM test.

6.2.1 Detailed Description

Interface and top-level testbench modules used by the verification environment.

6.3 Interfaces

Functions

- interface [s_interface](#) (input logic [clk](#), input logic reset)

6.3.1 Detailed Description

6.3.2 Function Documentation

6.3.2.1 s_interface()

```
interface s_interface (
    input logic clk,
    input logic reset)
```

< Pulsed high by the driver while a transaction is valid.

Modport used by the UVM driver: drives the stimulus signals.

Modport used by the UVM monitor: observes every signal, drives none.

Modport used by the register-file wrapper: consumes the stimulus and drives the reads.

6.4 UVM verification components

UVM classes and utilities used to verify the register file.

Files

- file [reference_model.c](#)
Golden reference model for the register file, called from the UVM scoreboard via DPI-C.
- file [agent.sv](#)
UVM agent that bundles the sequencer, driver, and monitor.
- file [coverage.sv](#)
Coverage collector for register-file transactions.
- file [driver.sv](#)
UVM driver that translates register-file transactions into interface signal activity.
- file [env.sv](#)
UVM environment that instantiates the agent, scoreboard, and coverage collector.
- file [monitor.sv](#)
UVM monitor that samples register-file transactions and publishes them to subscribers.
- file [pkt.sv](#)
Transaction item used to model one register-file access in the UVM environment.
- file [register_file_sequence.sv](#)
UVM sequence that drives register-file transactions until coverage reaches the target.
- file [scoreboard.sv](#)
Scoreboard that checks register-file transactions against the golden reference model.
- file [sequencer.sv](#)
UVM sequencer that provides transactions to the driver.
- file [test.sv](#)
Top-level UVM test that builds the environment and starts the main sequence.

6.4.1 Detailed Description

UVM classes and utilities used to verify the register file.

Chapter 7

Directory Documentation

7.1 filter Directory Reference

7.1.1 Detailed Description

7.1.2 SystemVerilog Doxygen filter

`idv_doxyfilter_sv.pl` converts SystemVerilog into C++-like code so Doxygen can parse it.

- **Author:** Sean O'Boyle, Intelligent Design Verification
- **Source:** <https://github.com/SeanOBoyle/DoxygenFilterSystemVerilog>
- **License:** GNU GPL v3 or later (see the header of the script). The file is included here unmodified.

It is wired up through `FILTER_PATTERNS` in `../Doxyfile`; run `doxygen` from the parent directory.

7.2 reference_model Directory Reference

Files

- file [reference_model.c](#)
Golden reference model for the register file, called from the UVM scoreboard via DPI-C.

7.3 rtl Directory Reference

Files

- file [register_file.v](#)
- file [register_file_wrapper.sv](#)
Wrapper that connects the register-file RTL to the UVM interface.

7.4 tb Directory Reference

Files

- file [s_interface.sv](#)
Register-file stimulus/response interface shared by the DUT and the UVM environment.
- file [tb_top.sv](#)
Top-level testbench that instantiates the DUT, interface, and UVM test.

7.5 uvm Directory Reference

Files

- file [agent.sv](#)
UVM agent that bundles the sequencer, driver, and monitor.
- file [coverage.sv](#)
Coverage collector for register-file transactions.
- file [driver.sv](#)
UVM driver that translates register-file transactions into interface signal activity.
- file [env.sv](#)
UVM environment that instantiates the agent, scoreboard, and coverage collector.
- file [monitor.sv](#)
UVM monitor that samples register-file transactions and publishes them to subscribers.
- file [pkt.sv](#)
Transaction item used to model one register-file access in the UVM environment.
- file [register_file_pkg.sv](#)
Package containing the register-file UVM verification components.
- file [register_file_sequence.sv](#)
UVM sequence that drives register-file transactions until coverage reaches the target.
- file [scoreboard.sv](#)
Scoreboard that checks register-file transactions against the golden reference model.
- file [sequencer.sv](#)
UVM sequencer that provides transactions to the driver.
- file [test.sv](#)
Top-level UVM test that builds the environment and starts the main sequence.

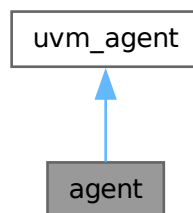
Chapter 8

Class Documentation

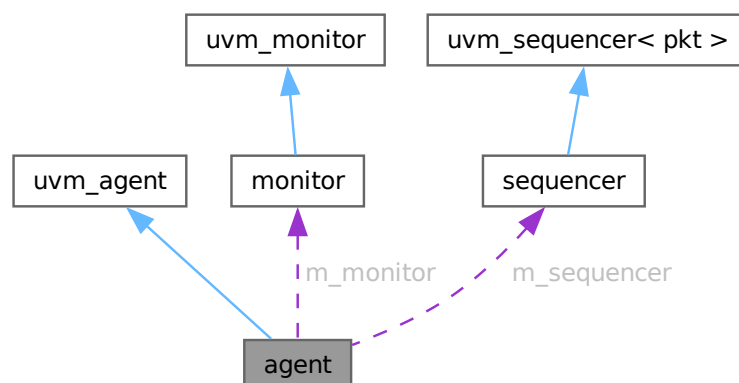
8.1 agent Class Reference

UVM agent that connects the sequencer, driver, and monitor.

Inheritance diagram for agent:



Collaboration diagram for agent:



Public Member Functions

- [uvm_component_utils](#) ([agent](#)) [driver](#) m_driver
Drives sequence items onto the DUT interface (null when passive).
- [new](#) (string name="agent", uvm_component parent=null)
- virtual void [build_phase](#) (uvm_phase phase)
Builds the sequencer/driver (active agent only) and the monitor.
- virtual void [connect_phase](#) (uvm_phase phase)
Connects the driver's seq_item port to the sequencer (active agent only).

Public Attributes

- [monitor](#) m_monitor
Samples DUT activity and publishes it (always created).
- [sequencer](#) m_sequencer
Arbitrates sequence items for the driver (null when passive).

8.1.1 Detailed Description

UVM agent that connects the sequencer, driver, and monitor.

Coordinates the active and passive verification components for the register-file testbench and exposes the common sequence and analysis interfaces. The sequencer and driver are only created when the agent is active; the monitor is always created.

8.1.2 Member Function Documentation

8.1.2.1 build_phase()

```
virtual void agent::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Builds the sequencer/driver (active agent only) and the monitor.

8.1.2.2 connect_phase()

```
virtual void agent::connect_phase (  
    uvm_phase phase) [inline], [virtual]
```

Connects the driver's seq_item port to the sequencer (active agent only).

8.1.2.3 new()

```
agent::new (  
    string name = "agent",  
    uvm_component parent = null) [inline]
```

8.1.2.4 uvm_component_utils()

```
agent::uvm_component_utils (
    agent )
```

Drives sequence items onto the DUT interface (null when passive).

8.1.3 Member Data Documentation

8.1.3.1 m_monitor

```
monitor agent::m_monitor
```

Samples DUT activity and publishes it (always created).

8.1.3.2 m_sequencer

```
sequencer agent::m_sequencer
```

Arbitrates sequence items for the driver (null when passive).

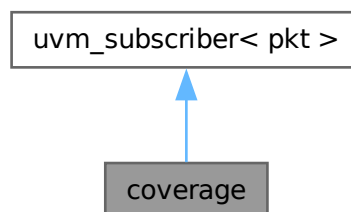
The documentation for this class was generated from the following file:

- [uvm/agent.sv](#)

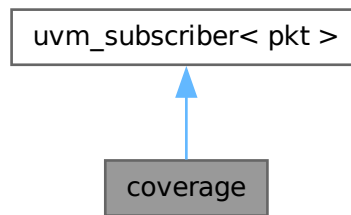
8.2 coverage Class Reference

UVM coverage subscriber for register-file transaction sampling.

Inheritance diagram for coverage:



Collaboration diagram for coverage:



Public Member Functions

- `uvm_component_utils (coverage) pkt tr`
Transaction currently being sampled by `cg_register_file`.
- `covergroup cg_register_file ()`
Write enable x write address x rs1 address x rs2 address cross coverage model.
- `new` (string name="coverage", uvm_component parent)
- virtual void `write (pkt t)`
Samples the covergroup for one transaction and republishes the running coverage percentage.

8.2.1 Detailed Description

UVM coverage subscriber for register-file transaction sampling.

Tracks the possible values of the write enable, write address, and both read addresses, including their direct cross coverage. Publishes the current percentage to the sequence layer through the configuration database under the key "cov_status".

8.2.2 Member Function Documentation

8.2.2.1 cg_register_file()

```
covergroup coverage::cg_register_file () [inline]
```

Write enable x write address x rs1 address x rs2 address cross coverage model.

Both write-enable states, so writes and non-writes are exercised.

One bin per register (x0..x15) as write destination; x0 writes must be ignored.

One bin per register read through rs1 (instr[18:15]).

One bin per register read through rs2 (instr[23:20]).

Full cross of write enable/address with both read addresses, so every read-after-write combination (including same-cycle bypass) is hit.

8.2.2.2 new()

```
coverage::new (
    string name = "coverage",
    uvm_component parent) [inline]
```

8.2.2.3 uvm_component_utils()

```
coverage::uvm_component_utils (
    coverage )
```

Transaction currently being sampled by [cg_register_file](#).

8.2.2.4 write()

```
virtual void coverage::write (
    pkt t) [inline], [virtual]
```

Samples the covergroup for one transaction and republishes the running coverage percentage.

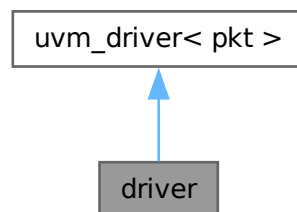
The documentation for this class was generated from the following file:

- [uvm/coverage.sv](#)

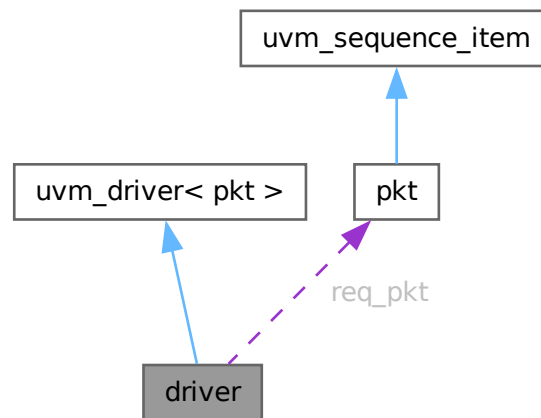
8.3 driver Class Reference

UVM driver for register-file transactions.

Inheritance diagram for driver:



Collaboration diagram for driver:



Public Member Functions

- [uvm_component_utils](#) ([driver](#)) virtual s_interface.driver [vif](#)
Driver modport of the DUT interface, obtained from the config DB (key "[vif](#)").
- [new](#) (string name="driver", uvm_component parent)
- virtual void [build_phase](#) (uvm_phase phase)
Fetches the virtual interface handle set by the top-level testbench.
- virtual task [run_phase](#) (uvm_phase phase)
Waits for reset to be released, then drives sequence items forever.
- virtual task [drive_item](#) ([pkt](#) [pkt_item](#))
Drives one full instruction-field/EX-feedback set for one clock cycle.

Public Attributes

- [pkt req_pkt](#)
Sequence item currently being driven.

8.3.1 Detailed Description

UVM driver for register-file transactions.

Converts each transaction item into the corresponding stimulus pulse on [s_interface](#). Requires the virtual interface to be set in the configuration database under the key "[vif](#)".

8.3.2 Member Function Documentation

8.3.2.1 build_phase()

```
virtual void driver::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fetches the virtual interface handle set by the top-level testbench.

8.3.2.2 drive_item()

```
virtual task driver::drive_item (  
    pkt pkt_item) [inline], [virtual]
```

Drives one full instruction-field/EX-feedback set for one clock cycle.

8.3.2.3 new()

```
driver::new (  
    string name = "driver",  
    uvm_component parent) [inline]
```

8.3.2.4 run_phase()

```
virtual task driver::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Waits for reset to be released, then drives sequence items forever.

8.3.2.5 uvm_component_utils()

```
driver::uvm_component_utils (  
    driver )
```

Driver modport of the DUT interface, obtained from the config DB (key "[vif](#)").

8.3.3 Member Data Documentation

8.3.3.1 req_pkt

```
pkt driver::req_pkt
```

Sequence item currently being driven.

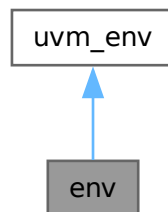
The documentation for this class was generated from the following file:

- [uvm/driver.sv](#)

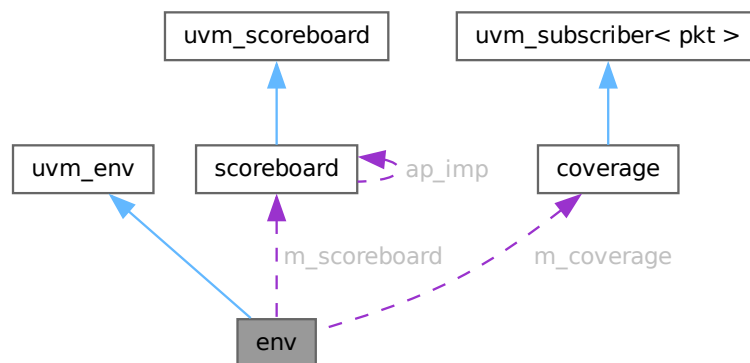
8.4 env Class Reference

Top-level UVM environment for the register-file verification bench.

Inheritance diagram for env:



Collaboration diagram for env:



Public Member Functions

- `uvm_component_utils (env) agent m_agent`
Agent that drives and monitors the DUT.
- `new (string name="env", uvm_component parent=null)`
- `virtual void build_phase (uvm_phase phase)`
Creates the agent, scoreboard, and coverage collector.
- `virtual void connect_phase (uvm_phase phase)`
Connects the monitor's analysis port to both the scoreboard and coverage collector.

Public Attributes

- [scoreboard m_scoreboard](#)
Checks monitored transactions against the reference model.
- [coverage m_coverage](#)
Collects functional coverage of monitored transactions.

8.4.1 Detailed Description

Top-level UVM environment for the register-file verification bench.

Instantiates the agent, scoreboard, and coverage collector and connects the monitored transactions to the checkers.

8.4.2 Member Function Documentation

8.4.2.1 build_phase()

```
virtual void env::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Creates the agent, scoreboard, and coverage collector.

8.4.2.2 connect_phase()

```
virtual void env::connect_phase (  
    uvm_phase phase) [inline], [virtual]
```

Connects the monitor's analysis port to both the scoreboard and coverage collector.

8.4.2.3 new()

```
env::new (  
    string name = "env",  
    uvm_component parent = null) [inline]
```

8.4.2.4 uvm_component_utils()

```
env::uvm_component_utils (  
    env )
```

Agent that drives and monitors the DUT.

8.4.3 Member Data Documentation

8.4.3.1 m_coverage

```
coverage env::m_coverage
```

Collects functional coverage of monitored transactions.

8.4.3.2 m_scoreboard

`scoreboard env::m_scoreboard`

Checks monitored transactions against the reference model.

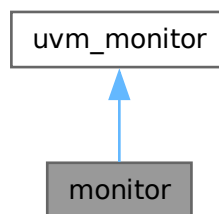
The documentation for this class was generated from the following file:

- [uvm/env.sv](#)

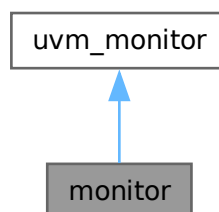
8.5 monitor Class Reference

UVM monitor for register-file transaction observation.

Inheritance diagram for monitor:



Collaboration diagram for monitor:



Public Member Functions

- [uvm_component_utils](#) ([monitor](#)) virtual `s_interface.monitor vif`
Monitor modport of the DUT interface, obtained from the config DB (key "[vif](#)").
- [new](#) (string name="monitor", `uvm_component` parent=null)
- virtual void [build_phase](#) (`uvm_phase` phase)
Fetches the virtual interface handle set by the top-level testbench.
- virtual task [run_phase](#) (`uvm_phase` phase)
Samples one register-file transaction per iteration and publishes it.

Public Attributes

- `uvm_analysis_port< pkt > mon_analysis_port`

Publishes each observed transaction to the scoreboard and coverage collector.

8.5.1 Detailed Description

UVM monitor for register-file transaction observation.

Samples the DUT-visible signals on each transfer and publishes the resulting transaction object to downstream analysis components. Requires the virtual interface to be set in the configuration database under the key "`vif`".

8.5.2 Member Function Documentation

8.5.2.1 build_phase()

```
virtual void monitor::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fetches the virtual interface handle set by the top-level testbench.

8.5.2.2 new()

```
monitor::new (  
    string name = "monitor",  
    uvm_component parent = null) [inline]
```

8.5.2.3 run_phase()

```
virtual task monitor::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Samples one register-file transaction per iteration and publishes it.

The register-file read ports update on the falling clock edge, so the monitor waits for that edge before sampling the outputs.

8.5.2.4 uvm_component_utils()

```
monitor::uvm_component_utils (  
    monitor )
```

Monitor modport of the DUT interface, obtained from the config DB (key "`vif`").

8.5.3 Member Data Documentation

8.5.3.1 mon_analysis_port

`uvm_analysis_port<pkt> monitor::mon_analysis_port`

Publishes each observed transaction to the scoreboard and coverage collector.

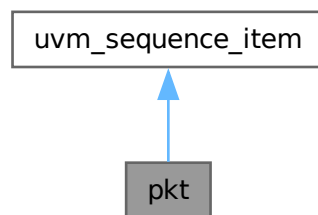
The documentation for this class was generated from the following file:

- `uvm/monitor.sv`

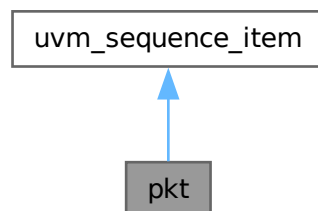
8.6 pkt Class Reference

Sequence item that models one register-file access.

Inheritance diagram for pkt:



Collaboration diagram for pkt:



Public Member Functions

- [uvm_object_utils_begin](#) (pkt) [uvm_field_int](#)([i_write_en_WB](#)
- UVM_DEFAULT [uvm_field_int](#) ([i_data_WB](#), UVM_DEFAULT) [uvm_field_int](#)([i_rd_WB](#)
- UVM_DEFAULT UVM_DEFAULT [uvm_field_int](#) ([i_instr_ID](#), UVM_DEFAULT) [uvm_field_int](#)([o_rs1_ID](#)

Public Attributes

- rand logic [i_write_en_WB](#)
Register-file write enable at WriteBack.
- rand logic< 31:0 > [i_data_WB](#)
Data to write at WriteBack.
- rand logic< 3:0 > [i_rd_WB](#)
Destination register address at WriteBack.
- rand logic< 31:0 > [i_instr_ID](#)
Instruction at Decode; rs1 = instr[18:15], rs2 = instr[23:20].
- logic< 31:0 > [o_rs1_ID](#)
Data read for rs1, sampled by the monitor.
- logic< 31:0 > [o_rs2_ID](#)
Data read for rs2, sampled by the monitor.

8.6.1 Detailed Description

Sequence item that models one register-file access.

Carries the fields randomized by the sequence/driver, and the result fields filled in by the monitor for the scoreboard to check against the golden reference model.

8.6.2 Member Function Documentation

8.6.2.1 [uvm_field_int\(\)](#) [1/2]

```
UVM_DEFAULT pkt::uvm_field_int (
    i\_data\_WB ,
    UVM_DEFAULT )
```

8.6.2.2 [uvm_field_int\(\)](#) [2/2]

```
UVM_DEFAULT UVM_DEFAULT pkt::uvm_field_int (
    i\_instr\_ID ,
    UVM_DEFAULT )
```

8.6.2.3 [uvm_object_utils_begin\(\)](#)

```
pkt::uvm_object_utils_begin (
    pkt )
```

8.6.3 Member Data Documentation

8.6.3.1 i_data_WB

```
rand logic<31:0> pkt::i_data_WB
```

Data to write at WriteBack.

8.6.3.2 i_instr_ID

```
rand logic<31:0> pkt::i_instr_ID
```

Instruction at Decode; rs1 = instr[18:15], rs2 = instr[23:20].

8.6.3.3 i_rd_WB

```
rand logic<3:0> pkt::i_rd_WB
```

Destination register address at WriteBack.

8.6.3.4 i_write_en_WB

```
rand logic pkt::i_write_en_WB
```

Register-file write enable at WriteBack.

8.6.3.5 o_rs1_ID

```
logic<31:0> pkt::o_rs1_ID
```

Data read for rs1, sampled by the monitor.

8.6.3.6 o_rs2_ID

```
logic<31:0> pkt::o_rs2_ID
```

Data read for rs2, sampled by the monitor.

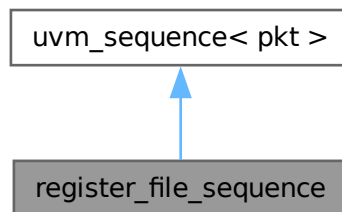
The documentation for this class was generated from the following file:

- [uvm/pkt.sv](#)

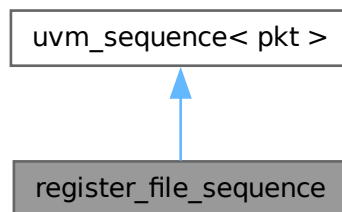
8.7 register_file_sequence Class Reference

UVM sequence that drives register-file transactions until coverage reaches the target.

Inheritance diagram for register_file_sequence:



Collaboration diagram for register_file_sequence:



Public Member Functions

- `uvms_object_utils (register_file_sequence)` real current_coverage=0
Latest coverage percentage reported by the coverage collector.
- `new` (string name="register_file_sequence")
- virtual task `body` ()
Sends randomized register-file transactions until the coverage collector reports 100%.

Public Attributes

- int `num_packets` = 0
Number of transactions sent so far in this sequence.

8.7.1 Detailed Description

UVM sequence that drives register-file transactions until coverage reaches the target.

Repeatedly randomizes and sends a transaction, polling the coverage collector after each one, until 100% coverage is reached. Reads the coverage percentage from the configuration database key "cov_status".

8.7.2 Member Function Documentation

8.7.2.1 body()

```
virtual task register_file_sequence::body () [inline], [virtual]
```

Sends randomized register-file transactions until the coverage collector reports 100%.

8.7.2.2 new()

```
register_file_sequence::new (
    string name = "register_file_sequence") [inline]
```

8.7.2.3 uvm_object_utils()

```
register_file_sequence::uvm_object_utils (
    register_file_sequence ) [pure virtual]
```

Latest coverage percentage reported by the coverage collector.

8.7.3 Member Data Documentation

8.7.3.1 num_packets

```
int register_file_sequence::num_packets = 0
```

Number of transactions sent so far in this sequence.

The documentation for this class was generated from the following file:

- [uvm/register_file_sequence.sv](#)

8.8 s_interface Interface Reference

Register-file stimulus/response interface shared by the DUT and the UVM environment.

8.8.1 Detailed Description

Register-file stimulus/response interface shared by the DUT and the UVM environment.

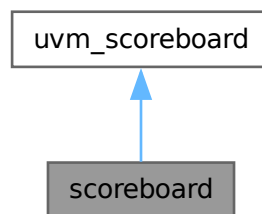
The documentation for this interface was generated from the following file:

- [tb/s_interface.sv](#)

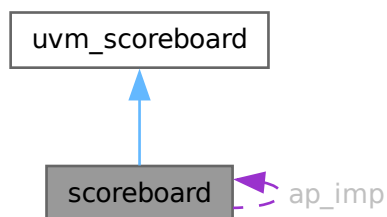
8.9 scoreboard Class Reference

UVM scoreboard for register-file functional checking.

Inheritance diagram for scoreboard:



Collaboration diagram for scoreboard:



Public Member Functions

- [uvm_component_utils](#) (scoreboard) uvm_analysis_imp< [pkt](#)
- [new](#) (string name="scoreboard", uvm_component parent=null)
- virtual void [build_phase](#) (uvm_phase phase)
Creates the analysis imp that receives transactions from the monitor.
- virtual void [write](#) ([pkt](#) data)
Analysis-port callback: checks one monitored transaction against [register_file_golden](#).
- virtual void [check_phase](#) (uvm_phase phase)
Fails the test if any mismatch was recorded by [write\(\)](#).

Public Attributes

- [scoreboard ap_imp](#)
Receives transactions from the monitor's analysis port.
- `int num_errors = 0`
Number of mismatches found so far.

8.9.1 Detailed Description

UVM scoreboard for register-file functional checking.

Compares the monitored read data with the DPI-C reference model and reports mismatches as test failures.

8.9.2 Member Function Documentation

8.9.2.1 build_phase()

```
virtual void scoreboard::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Creates the analysis imp that receives transactions from the monitor.

8.9.2.2 check_phase()

```
virtual void scoreboard::check_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fails the test if any mismatch was recorded by `write()`.

8.9.2.3 new()

```
scoreboard::new (  
    string name = "scoreboard",  
    uvm_component parent = null) [inline]
```

8.9.2.4 uvm_component_utils()

```
scoreboard::uvm_component_utils (  
    scoreboard )
```

8.9.2.5 write()

```
virtual void scoreboard::write (  
    pkt data) [inline], [virtual]
```

Analysis-port callback: checks one monitored transaction against `register_file_golden`.

8.9.3 Member Data Documentation

8.9.3.1 ap_imp

`scoreboard scoreboard::ap_imp`

Receives transactions from the monitor's analysis port.

8.9.3.2 num_errors

`int scoreboard::num_errors = 0`

Number of mismatches found so far.

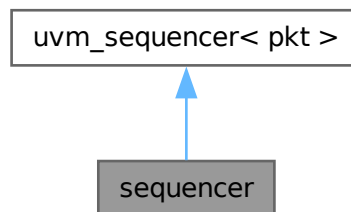
The documentation for this class was generated from the following file:

- [uvm/scoreboard.sv](#)

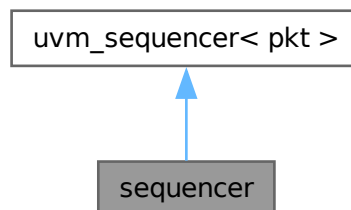
8.10 sequencer Class Reference

UVM sequencer for register-file transactions.

Inheritance diagram for sequencer:



Collaboration diagram for sequencer:



Public Member Functions

- [uvm_component_utils](#) ([sequencer](#)) new(string name

Public Attributes

- uvm_component [parent](#)

8.10.1 Detailed Description

UVM sequencer for register-file transactions.

Supplies randomized sequence items to the driver during the test run.

8.10.2 Member Function Documentation

8.10.2.1 [uvm_component_utils\(\)](#)

```
sequencer::uvm_component_utils (  
    sequencer ) [new]
```

8.10.3 Member Data Documentation

8.10.3.1 [parent](#)

```
uvm_component sequencer::parent
```

Initial value:

```
{  
    super.new (name, parent)
```

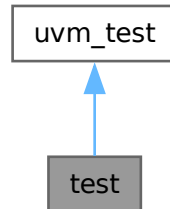
The documentation for this class was generated from the following file:

- uvm/[sequencer.sv](#)

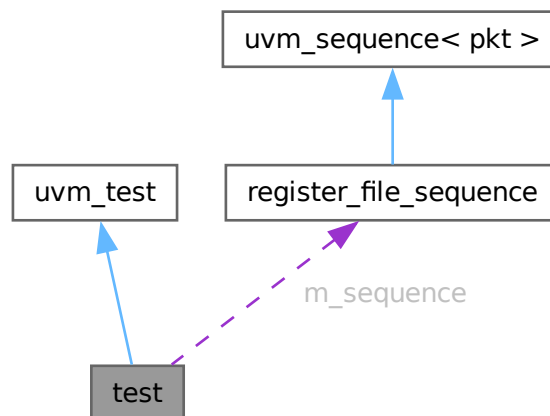
8.11 test Class Reference

Top-level UVM test for the register-file verification bench.

Inheritance diagram for test:



Collaboration diagram for test:



Public Member Functions

- `uvm_component_utils (test) env m_env`
Verification environment under test.
- `new` (string name="test", uvm_component parent=null)
- virtual void `build_phase` (uvm_phase phase)
Builds the verification environment.
- virtual void `end_of_elaboration_phase` (uvm_phase phase)
Prints the component topology once elaboration is complete.
- virtual task `run_phase` (uvm_phase phase)
Starts the coverage-driven sequence and raises/drops the run-phase objection around it.

Public Attributes

- [register_file_sequence m_sequence](#)
Main stimulus sequence, started on the agent's sequencer.

8.11.1 Detailed Description

Top-level UVM test for the register-file verification bench.

Builds the environment, starts the main sequence, and drives the full verification flow from the test root.

8.11.2 Member Function Documentation

8.11.2.1 build_phase()

```
virtual void test::build_phase (
    uvm_phase phase) [inline], [virtual]
```

Builds the verification environment.

8.11.2.2 end_of_elaboration_phase()

```
virtual void test::end_of_elaboration_phase (
    uvm_phase phase) [inline], [virtual]
```

Prints the component topology once elaboration is complete.

8.11.2.3 new()

```
test::new (
    string name = "test",
    uvm_component parent = null) [inline]
```

8.11.2.4 run_phase()

```
virtual task test::run_phase (
    uvm_phase phase) [inline], [virtual]
```

Starts the coverage-driven sequence and raises/drops the run-phase objection around it.

8.11.2.5 uvm_component_utils()

```
test::uvm_component_utils (
    test )
```

Verification environment under test.

8.11.3 Member Data Documentation

8.11.3.1 m_sequence

```
register\_file\_sequence test::m_sequence
```

Main stimulus sequence, started on the agent's sequencer.

The documentation for this class was generated from the following file:

- [uvm/test.sv](#)

Chapter 9

File Documentation

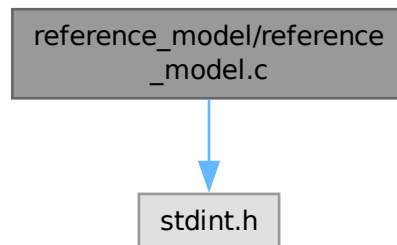
9.1 filter/README.md File Reference

9.2 reference_model/reference_model.c File Reference

Golden reference model for the register file, called from the UVM scoreboard via DPI-C.

```
#include <stdint.h>
```

Include dependency graph for reference_model.c:



Macros

- `#define NUM_REGS 16u`

Functions

- void `register_file_golden` (unsigned char write_en, unsigned int write_data, unsigned char write_addr, unsigned int instr, unsigned int *exp_rs1, unsigned int *exp_rs2)

Applies one write and returns the expected values of the two read ports.

9.2.1 Detailed Description

Golden reference model for the register file, called from the UVM scoreboard via DPI-C.

Keeps a software copy of the 16 registers, with the same reset values as the RTL (register *i* holds *i*), and models the write-then-read behavior of one transaction. Register x0 is hard-wired to zero: writes to it are ignored and reads of it return 0.

9.2.2 Macro Definition Documentation

9.2.2.1 NUM_REGS

```
#define NUM_REGS 16u
```

9.2.3 Function Documentation

9.2.3.1 register_file_golden()

```
void register_file_golden (
    unsigned char write_en,
    unsigned int write_data,
    unsigned char write_addr,
    unsigned int instr,
    unsigned int * exp_rs1,
    unsigned int * exp_rs2)
```

Applies one write and returns the expected values of the two read ports.

The write is applied before the reads, so a read of the register just written returns the new data (the RTL writes on the rising edge and reads on the falling edge of the same cycle). State persists across calls.

Parameters

<i>write_en</i>	1 to write <i>write_data</i> into <i>write_addr</i> .
<i>write_data</i>	Data to write.
<i>write_addr</i>	Destination register address (writes to x0 are ignored).
<i>instr</i>	Instruction; rs1 = <i>instr</i> [18:15], rs2 = <i>instr</i> [23:20].
<i>exp_rs1</i>	Output: expected o_rs1_ID .
<i>exp_rs2</i>	Output: expected o_rs2_ID .

9.3 rtl/register_file.v File Reference

Functions

- [always](#) (posedge [clk](#)) begin if([i_rst_ID](#)) begin for(integer i=0

Variables

- input logic [clk](#)
- input logic [i_rst_ID](#)
- input logic [i_write_en_WB](#)
- input logic[DATA_WIDTH-1:0] [i_data_WB](#)
- input logic[INDEX_WIDTH-1:0] [i_rd_WB](#)
- input logic[DATA_WIDTH-1:0] [i_instr_ID](#)
- output logic[DATA_WIDTH-1:0] [o_rs1_ID](#)
- output logic[DATA_WIDTH-1:0] [o_rs2_ID](#)
- reg[DATA_WIDTH-1:0] [registers](#) [15:0]

9.3.1 Function Documentation

9.3.1.1 always()

```
always (
    posedge clk) [pure virtual]
```

9.3.2 Variable Documentation

9.3.2.1 clk

```
input logic clk
```

9.3.2.2 i_data_WB

```
input logic [DATA_WIDTH-1:0] i_data_WB
```

9.3.2.3 i_instr_ID

```
input logic [DATA_WIDTH-1:0] i_instr_ID
```

9.3.2.4 i_rd_WB

```
input logic [INDEX_WIDTH-1:0] i_rd_WB
```

9.3.2.5 i_rst_ID

```
input logic i_rst_ID
```

9.3.2.6 i_write_en_WB

```
input logic i_write_en_WB
```

9.3.2.7 o_rs1_ID

```
output logic [DATA_WIDTH-1:0] o_rs1_ID
```

9.3.2.8 o_rs2_ID

```
output logic [DATA_WIDTH-1:0] o_rs2_ID
```

9.3.2.9 registers

```
reg [DATA_WIDTH-1:0] registers[15:0]
```

9.4 rtl/register_file_wrapper.sv File Reference

Wrapper that connects the register-file RTL to the UVM interface.

9.4.1 Detailed Description

Wrapper that connects the register-file RTL to the UVM interface.

9.5 tb/s_interface.sv File Reference

Register-file stimulus/response interface shared by the DUT and the UVM environment.

Functions

- interface [s_interface](#) (input logic [clk](#), input logic reset)

9.5.1 Detailed Description

Register-file stimulus/response interface shared by the DUT and the UVM environment.

Keeps the existing clock/reset/enable transaction protocol while exposing register-file write inputs and two read-data outputs.

9.6 tb/tb_top.sv File Reference

Top-level testbench that instantiates the DUT, interface, and UVM test.

Functions

- `s_interface vif (.clk(clk),.reset(rst))`
- `register_file_wrapper U_REGISTER_FILE_WRAPPER (.vif(vif))`
- `run_test ("test")`

Variables

- bit `rst`
- `forever`

9.6.1 Detailed Description

Top-level testbench that instantiates the DUT, interface, and UVM test.

Connects `s_interface`, the `register_file_wrapper` DUT, and the UVM test entry point in a single simulation environment.

9.6.2 Function Documentation

9.6.2.1 run_test()

```
run_test (
    "test" )
```

9.6.2.2 U_REGISTER_FILE_WRAPPER()

```
register_file_wrapper U_REGISTER_FILE_WRAPPER (
    . vifvif)
```

9.6.2.3 vif()

```
s_interface vif (
    . clkclk,
    . resetrst)
```

9.6.3 Variable Documentation

9.6.3.1 forever

```
forever
```

Initial value:

```
{
    rst = 0
```

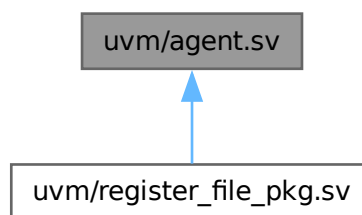
9.6.3.2 rst

```
bit rst
```

9.7 uvm/agent.sv File Reference

UVM agent that bundles the sequencer, driver, and monitor.

This graph shows which files directly or indirectly include this file:



Classes

- class [agent](#)
UVM agent that connects the sequencer, driver, and monitor.

9.7.1 Detailed Description

UVM agent that bundles the sequencer, driver, and monitor.

Implements the standard UVM agent structure for the register-file verification environment, creating active and passive components as needed.

9.8 agent.sv

[Go to the documentation of this file.](#)

```

00001
00018 class agent extends uvm_agent;
00019     `uvm_component_utils (agent)
00020
00021     driver m_driver;
00022
00023     monitor m_monitor;
00024
00025     sequencer m_sequencer;
00026
00027     function new (string name = "agent", uvm_component parent = null);
00028         super.new (name, parent);
00029     endfunction
  
```

```

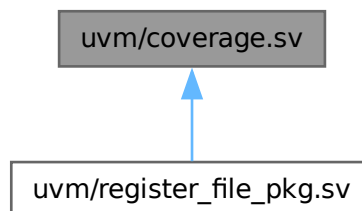
00030
00032     virtual function void build_phase (uvm_phase phase);
00033         super.build_phase (phase);
00034
00035         if (get_is_active()) begin
00036             m_sequencer = sequencer::type_id::create ("sequencer", this);
00037             m_driver = driver::type_id::create ("driver", this);
00038         end
00039
00040         m_monitor = monitor::type_id::create ("monitor", this);
00041
00042     endfunction
00043
00045     virtual function void connect_phase (uvm_phase phase);
00046         super.connect_phase (phase);
00047         if (get_is_active()) begin
00048             m_driver.seq_item_port.connect (m_sequencer.seq_item_export);
00049         end
00050     endfunction
00051
00052 endclass

```

9.9 uvm/coverage.sv File Reference

Coverage collector for register-file transactions.

This graph shows which files directly or indirectly include this file:



Classes

- class `coverage`
UVM coverage subscriber for register-file transaction sampling.

9.9.1 Detailed Description

Coverage collector for register-file transactions.

Samples observed register-file transactions and updates the coverage model used to determine whether the verification goal has been reached.

9.10 coverage.sv

[Go to the documentation of this file.](#)

```

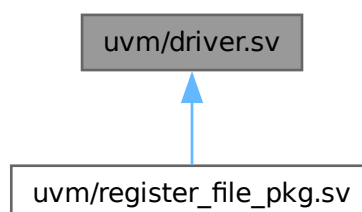
00001
00018 class coverage extends uvm_subscriber #(pkt);
00019     `uvm_component_utils(coverage)
00020
00021     pkt tr;
00022
00024     covergroup cg_register_file;
00025         option.per_instance = 1;
00026
00028         cp_write_enable: coverpoint tr.i_write_en_WB {
00029             bins disabled = {0};
00030             bins enabled = {1};
00031         }
00032
00034         cp_write_address: coverpoint tr.i_rd_WB {
00035             bins registers[16] = {[0:15]};
00036         }
00037
00039         cp_rd1_address: coverpoint tr.i_instr_ID[18:15] {
00040             bins registers[16] = {[0:15]};
00041         }
00042
00044         cp_rd2_address: coverpoint tr.i_instr_ID[23:20] {
00045             bins registers[16] = {[0:15]};
00046         }
00047
00050         cross_register_file: cross cp_write_enable, cp_write_address, cp_rd1_address, cp_rd2_address;
00051
00052     endgroup
00053
00054     function new(string name = "coverage", uvm_component parent);
00055         super.new(name, parent);
00056         cg_register_file = new();
00057         cg_register_file.set_inst_name("register_file_cov");
00058     endfunction
00059
00061     virtual function void write(pkt t);
00062         this.tr = t;
00063         cg_register_file.sample();
00064         uvm_config_db#(real)::set(null, "*", "cov_status", cg_register_file.get_inst_coverage());
00065     endfunction
00066
00067 endclass

```

9.11 uvm/driver.sv File Reference

UVM driver that translates register-file transactions into interface signal activity.

This graph shows which files directly or indirectly include this file:



Classes

- class [driver](#)

UVM driver for register-file transactions.

9.11.1 Detailed Description

UVM driver that translates register-file transactions into interface signal activity.

Receives sequence items from the sequencer and drives the instruction fields/EX-stage feedback onto [s_interface](#) for one clock cycle each.

9.12 driver.sv

[Go to the documentation of this file.](#)

```

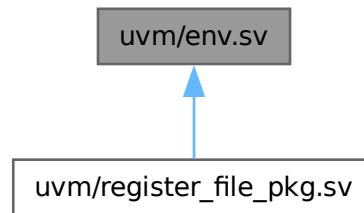
00001
00017 class driver extends uvm_driver #(pkt);
00018     `uvm_component_utils (driver)
00019
00020     virtual s_interface.driver vif;
00021
00022     pkt req_pkt;
00023
00024     function new (string name = "driver", uvm_component parent);
00025         super.new(name, parent);
00026     endfunction
00027
00029     virtual function void build_phase (uvm_phase phase);
00030         super.build_phase(phase);
00031         if (!uvm_config_db #(virtual s_interface)::get(this, "", "vif", vif)) begin
00032             `uvm_fatal (get_type_name (), "Didn't get handle to virtual interface if_name")
00033         end
00034     endfunction
00035
00037     virtual task run_phase(uvm_phase phase);
00038         super.run_phase(phase);
00039
00040         wait (vif.reset == 1'b1);
00041
00042         forever begin
00043             seq_item_port.get_next_item(req_pkt);
00044             drive_item(req_pkt);
00045             seq_item_port.item_done();
00046         end
00047     endtask
00048
00050     virtual task drive_item (pkt pkt_item);
00051         @(negedge vif.clk);
00052         vif.write_en    <= pkt_item.i_write_en_WB;
00053         vif.write_data  <= pkt_item.i_data_WB;
00054         vif.write_addr  <= pkt_item.i_rd_WB;
00055         vif.instr       <= pkt_item.i_instr_ID;
00056         vif.enable      <= '1;
00057
00058         @(negedge vif.clk);
00059         vif.enable <= '0;
00060     endtask
00061
00062 endclass

```

9.13 uvm/env.sv File Reference

UVM environment that instantiates the agent, scoreboard, and coverage collector.

This graph shows which files directly or indirectly include this file:



Classes

- class `env`

Top-level UVM environment for the register-file verification bench.

9.13.1 Detailed Description

UVM environment that instantiates the agent, scoreboard, and coverage collector.

Connects the main verification components so that transactions generated by the agent can be checked and tracked by the scoreboard and coverage.

9.14 env.sv

[Go to the documentation of this file.](#)

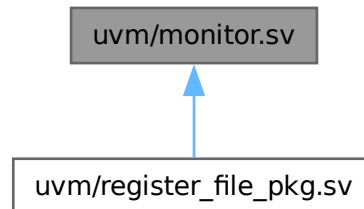
```

00001
00016 class env extends uvm_env;
00017     `uvm_component_utils (env)
00018
00019     agent m_agent;
00020
00021     scoreboard m_scoreboard;
00022
00023     coverage m_coverage;
00024
00025     function new (string name = "env", uvm_component parent = null);
00026         super.new (name, parent);
00027     endfunction
00028
00030     virtual function void build_phase (uvm_phase phase);
00031         super.build_phase (phase);
00032         m_agent = agent::type_id::create ("agent", this);
00033         m_scoreboard = scoreboard::type_id::create ("scoreboard", this);
00034         m_coverage = coverage::type_id::create ("coverage", this);
00035     endfunction
00036
00038     virtual function void connect_phase (uvm_phase phase);
00039         super.connect_phase (phase);
00040         m_agent.m_monitor.mon_analysis_port.connect (m_scoreboard.ap_imp);
00041         m_agent.m_monitor.mon_analysis_port.connect (m_coverage.analysis_export);
00042     endfunction
00043
00044 endclass
  
```

9.15 uvm/monitor.sv File Reference

UVM monitor that samples register-file transactions and publishes them to subscribers.

This graph shows which files directly or indirectly include this file:



Classes

- class [monitor](#)
UVM monitor for register-file transaction observation.

9.15.1 Detailed Description

UVM monitor that samples register-file transactions and publishes them to subscribers.

Observes [s_interface](#) for each transaction and forwards the completed input/output fields to the scoreboard and coverage collector.

9.16 monitor.sv

[Go to the documentation of this file.](#)

```

00001
00018 class monitor extends uvm_monitor;
00019     `uvm_component_utils(monitor)
00020
00021     virtual s_interface.monitor vif;
00022
00023     uvm_analysis_port #(pkt) mon_analysis_port;
00024
00025     function new(string name = "monitor", uvm_component parent = null);
00026         super.new(name, parent);
00027     endfunction
00028
00030     virtual function void build_phase(uvm_phase phase);
00031         super.build_phase(phase);
00032         mon_analysis_port = new("mon_analysis_port", this);
00033
00034         if (!uvm_config_db #(virtual s_interface)::get(this, "", "vif", vif)) begin
00035             `uvm_error(get_type_name(), "DUT interface not found")
00036         end
00037     endfunction
00038
00045     virtual task run_phase(uvm_phase phase);
00046         super.run_phase(phase);
  
```

```

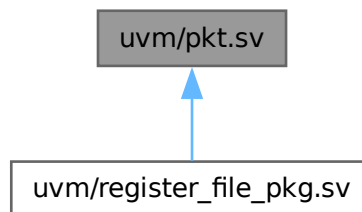
00047
00048     forever begin
00049         pkt mon_pkt;
00050         mon_pkt = pkt::type_id::create("mon_pkt", this);
00051
00052         @(posedge vif.enable);
00053
00054         #1;
00055
00056         mon_pkt.i_write_en_WB = vif.write_en;
00057         mon_pkt.i_data_WB      = vif.write_data;
00058         mon_pkt.i_rd_WB        = vif.write_addr;
00059         mon_pkt.i_instr_ID      = vif.instr;
00060
00061         @(negedge vif.clk);
00062         #1;
00063
00064         mon_pkt.o_rs1_ID       = vif.rs1_data;
00065         mon_pkt.o_rs2_ID       = vif.rs2_data;
00066
00067         `uvm_info(get_type_name(),
00068             $sformatf("Monitored write_en=%0b write_addr=%0d write_data=%08h instr=%08h ->
rs1=%08h rs2=%08h",
00069                 mon_pkt.i_write_en_WB, mon_pkt.i_rd_WB, mon_pkt.i_data_WB,
00070                 mon_pkt.i_instr_ID, mon_pkt.o_rs1_ID, mon_pkt.o_rs2_ID),
00071             UVM_LOW)
00072
00073         mon_analysis_port.write(mon_pkt);
00074     end
00075 endtask
00076
00077 endclass

```

9.17 uvm/pkt.sv File Reference

Transaction item used to model one register-file access in the UVM environment.

This graph shows which files directly or indirectly include this file:



Classes

- class `pkt`
Sequence item that models one register-file access.

9.17.1 Detailed Description

Transaction item used to model one register-file access in the UVM environment.

Encapsulates the write-back write port and the instruction whose rs1/rs2 fields select the registers to read, plus the read data sampled back from the register file.

9.18 pkt.sv

[Go to the documentation of this file.](#)

```

00001
00018 class pkt extends uvm_sequence_item;
00019
00020     rand logic         i_write_en_WB;
00021     rand logic [31:0] i_data_WB;
00022     rand logic [3:0]  i_rd_WB;
00023     rand logic [31:0] i_instr_ID;
00024
00025     logic [31:0] o_rs1_ID;
00026     logic [31:0] o_rs2_ID;
00027
00028     `uvm_object_utils_begin(pkt)
00029     `uvm_field_int (i_write_en_WB, UVM_DEFAULT)
00030     `uvm_field_int (i_data_WB, UVM_DEFAULT)
00031     `uvm_field_int (i_rd_WB, UVM_DEFAULT)
00032     `uvm_field_int (i_instr_ID, UVM_DEFAULT)
00033     `uvm_field_int (o_rs1_ID, UVM_DEFAULT)
00034     `uvm_field_int (o_rs2_ID, UVM_DEFAULT)
00035     `uvm_object_utils_end
00036
00037     function new(string name = "pkt");
00038         super.new(name);
00039     endfunction
00040
00041 endclass

```

9.19 uvm/register_file_pkg.sv File Reference

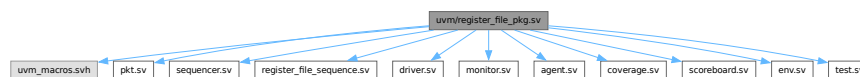
Package containing the register-file UVM verification components.

```

#include "uvm_macros.svh"
#include "pkt.sv"
#include "sequencer.sv"
#include "register_file_sequence.sv"
#include "driver.sv"
#include "monitor.sv"
#include "agent.sv"
#include "coverage.sv"
#include "scoreboard.sv"
#include "env.sv"
#include "test.sv"

```

Include dependency graph for register_file_pkg.sv:



9.19.1 Detailed Description

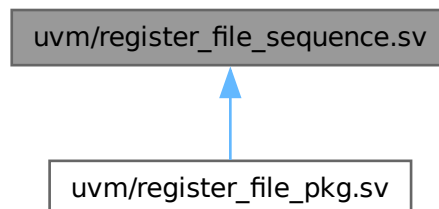
Package containing the register-file UVM verification components.

Centralizes the inclusion of all verification components required to build the register-file UVM testbench.

9.20 uvm/register_file_sequence.sv File Reference

UVM sequence that drives register-file transactions until coverage reaches the target.

This graph shows which files directly or indirectly include this file:



Classes

- class [register_file_sequence](#)

UVM sequence that drives register-file transactions until coverage reaches the target.

9.20.1 Detailed Description

UVM sequence that drives register-file transactions until coverage reaches the target.

Repeatedly sends randomized transactions while waiting for the coverage collector to report completion of the sampling process.

9.21 register_file_sequence.sv

[Go to the documentation of this file.](#)

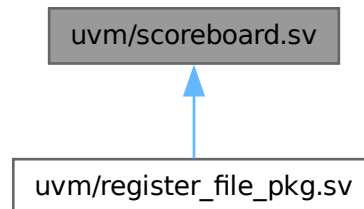
```

00001
00017 class register_file_sequence extends uvm_sequence #(pkt);
00018     `uvm_object_utils(register_file_sequence)
00019
00020     real current_coverage = 0;
00021
00022     int num_packets = 0;
00023
00024     function new (string name = "register_file_sequence");
00025         super.new(name);
00026     endfunction
00027
00029     virtual task body();
00030         pkt packet;
00031         `uvm_create(packet)
00032         while (current_coverage < 100.0) begin
00033             `uvm_rand_send(packet)
00034             num_packets++;
00035             void' (uvm_config_db#(real)::get(null, "*", "cov_status", current_coverage));
00036             `uvm_info("SEQ", $sformatf("Status: %0.2f%%", current_coverage), UVM_LOW)
00037         end
00038         `uvm_info("SEQ", $sformatf("Total packets sent: %0d", num_packets), UVM_LOW)
00039     endtask
00040 endclass
  
```

9.22 uvm/scoreboard.sv File Reference

Scoreboard that checks register-file transactions against the golden reference model.

This graph shows which files directly or indirectly include this file:



Classes

- class [scoreboard](#)
UVM scoreboard for register-file functional checking.

Functions

- void [register_file_golden](#) (input byte unsigned write_en, input int unsigned write_data, input byte unsigned write_addr, input int unsigned instr, output int unsigned exp_rs1, output int unsigned exp_rs2)
DPI-C import of the golden reference model (see [reference_model.c](#)).

9.22.1 Detailed Description

Scoreboard that checks register-file transactions against the golden reference model.

Receives monitored transactions and validates them against the DPI-C reference model implemented in the C source file.

9.22.2 Function Documentation

9.22.2.1 register_file_golden()

```
void register_file_golden (  
    input byte unsigned write_en,  
    input int unsigned write_data,  
    input byte unsigned write_addr,  
    input int unsigned instr,  
    output int unsigned exp_rs1,  
    output int unsigned exp_rs2)
```

DPI-C import of the golden reference model (see [reference_model.c](#)).

The model is stateful: it keeps its own copy of the registers, so it must be called once per transaction, in order.

Parameters

<i>write_en</i>	Register-file write enable.
<i>write_data</i>	Data to write.
<i>write_addr</i>	Destination register address.
<i>instr</i>	Instruction whose rs1/rs2 fields select the registers to read.
<i>exp_rs1</i>	Output: expected o_rs1_ID .
<i>exp_rs2</i>	Output: expected o_rs2_ID .

9.23 scoreboard.sv

[Go to the documentation of this file.](#)

```

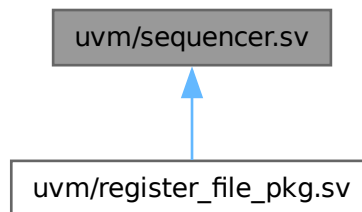
00001
00022 import "DPI-C" function void register_file_golden(
00023     input byte unsigned write_en,
00024     input int unsigned write_data,
00025     input byte unsigned write_addr,
00026     input int unsigned instr,
00027     output int unsigned exp_rs1,
00028     output int unsigned exp_rs2
00029 );
00030
00038 class scoreboard extends uvm_scoreboard;
00039     `uvm_component_utils(scoreboard)
00040
00041     uvm_analysis_imp #(pkt, scoreboard) ap_imp;
00042
00043     int num_errors = 0;
00044
00045     function new(string name = "scoreboard", uvm_component parent = null);
00046         super.new(name, parent);
00047     endfunction
00048
00050     virtual function void build_phase(uvm_phase phase);
00051         super.build_phase(phase);
00052         ap_imp = new("ap_imp", this);
00053     endfunction
00054
00056     virtual function void write(pkt data);
00057         int unsigned exp_rs1;
00058         int unsigned exp_rs2;
00059
00060         register_file_golden(data.i_write_en_WB, data.i_data_WB, data.i_rd_WB,
00061             data.i_instr_ID, exp_rs1, exp_rs2);
00062
00063         if (data.o_rs1_ID == exp_rs1 && data.o_rs2_ID == exp_rs2) begin
00064             `uvm_info("SCOREBOARD",
00065                 $sformatf("PASS: write_en=%0b write_addr=%0d write_data=%08h instr=%08h -> rs1=%08h rs2=%08h",
00066                     data.i_write_en_WB, data.i_rd_WB, data.i_data_WB, data.i_instr_ID,
00067                     data.o_rs1_ID, data.o_rs2_ID), UVM_LOW)
00068         end else begin
00069             `uvm_error("SCOREBOARD",
00070                 $sformatf("FAIL: write_en=%0b write_addr=%0d write_data=%08h instr=%08h EXPECTED rs1=%08h
00071 rs2=%08h GOT rs1=%08h rs2=%08h",
00072                     data.i_write_en_WB, data.i_rd_WB, data.i_data_WB, data.i_instr_ID,
00073                     exp_rs1, exp_rs2, data.o_rs1_ID, data.o_rs2_ID))
00074             num_errors++;
00075         end
00076     endfunction
00077
00078     virtual function void check_phase(uvm_phase phase);
00079         super.check_phase(phase);
00080         if (num_errors > 0) begin
00081             `uvm_fatal("FINAL_RESULT",
00082                 $sformatf("TEST FAILED: Scoreboard found %0d errors.", num_errors))
00083         end else begin
00084             `uvm_info("FINAL_RESULT", "TEST PASS: All transactions were correct.", UVM_NONE)
00085         end
00086     endfunction
00087
00088 endclass

```

9.24 uvm/sequencer.sv File Reference

UVM sequencer that provides transactions to the driver.

This graph shows which files directly or indirectly include this file:



Classes

- class [sequencer](#)
UVM sequencer for register-file transactions.

9.24.1 Detailed Description

UVM sequencer that provides transactions to the driver.

Acts as the transaction source for the UVM driver in the register-file verification.

9.25 sequencer.sv

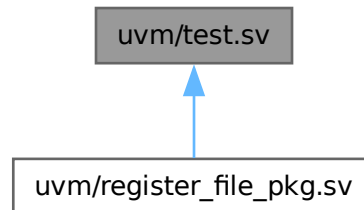
[Go to the documentation of this file.](#)

```
00001
00014 class sequencer extends uvm_sequencer #(pkt);
00015
00016     `uvm_component_utils (sequencer)
00017
00018     function new (string name="sequencer", uvm_component parent);
00019         super.new (name, parent);
00020     endfunction
00021
00022 endclass
```

9.26 uvm/test.sv File Reference

Top-level UVM test that builds the environment and starts the main sequence.

This graph shows which files directly or indirectly include this file:



Classes

- class `test`

Top-level UVM test for the register-file verification bench.

9.26.1 Detailed Description

Top-level UVM test that builds the environment and starts the main sequence.

Instantiates the verification environment and launches the register-file sequence that drives the DUT under test.

9.27 test.sv

[Go to the documentation of this file.](#)

```

00001
00016 class test extends uvm_test;
00017     `uvm_component_utils (test)
00018
00019     env m_env;
00020
00021     register_file_sequence m_sequence;
00022
00023     function new (string name = "test", uvm_component parent = null);
00024         super.new (name, parent);
00025     endfunction
00026
00028     virtual function void build_phase (uvm_phase phase);
00029         super.build_phase (phase);
00030         m_env = env::type_id::create ("env", this);
00031     endfunction
00032
00034     virtual function void end_of_elaboration_phase (uvm_phase phase);
00035         uvm_root::get().print_topology();
00036     endfunction
00037
00039     virtual task run_phase(uvm_phase phase);
00040         super.run_phase(phase);
00041
00042         m_sequence = register_file_sequence::type_id::create("sequence");
00043
00044         phase.raise_objection(this);
00045         m_sequence.start(m_env.m_agent.m_sequencer);
00046         phase.drop_objection(this);
00047     endtask
00048 endclass
  
```

Index

agent, [15](#)
 build_phase, [16](#)
 connect_phase, [16](#)
 m_monitor, [17](#)
 m_sequencer, [17](#)
 new, [16](#)
 uvm_component_utils, [16](#)

always
 register_file.v, [39](#)

ap_imp
 scoreboard, [33](#)

body
 register_file_sequence, [30](#)

build_phase
 agent, [16](#)
 driver, [21](#)
 env, [23](#)
 monitor, [25](#)
 scoreboard, [32](#)
 test, [36](#)

cg_register_file
 coverage, [18](#)

check_phase
 scoreboard, [32](#)

clk
 register_file.v, [39](#)

connect_phase
 agent, [16](#)
 env, [23](#)

coverage, [17](#)
 cg_register_file, [18](#)
 new, [18](#)
 uvm_component_utils, [19](#)
 write, [19](#)

drive_item
 driver, [21](#)

driver, [19](#)
 build_phase, [21](#)
 drive_item, [21](#)
 new, [21](#)
 req_pkt, [21](#)
 run_phase, [21](#)
 uvm_component_utils, [21](#)

end_of_elaboration_phase
 test, [36](#)

env, [22](#)
 build_phase, [23](#)
 connect_phase, [23](#)
 m_coverage, [23](#)
 m_scoreboard, [23](#)
 new, [23](#)
 uvm_component_utils, [23](#)

filter Directory Reference, [13](#)
filter/README.md, [37](#)

forever
 tb_top.sv, [41](#)

i_data_WB
 pkt, [28](#)
 register_file.v, [39](#)

i_instr_ID
 pkt, [28](#)
 register_file.v, [39](#)

i_rd_WB
 pkt, [28](#)
 register_file.v, [39](#)

i_rst_ID
 register_file.v, [39](#)

i_write_en_WB
 pkt, [28](#)
 register_file.v, [39](#)

Interfaces, [11](#)
 s_interface, [12](#)

m_coverage
 env, [23](#)

m_monitor
 agent, [17](#)

m_scoreboard
 env, [23](#)

m_sequence
 test, [36](#)

m_sequencer
 agent, [17](#)

Modules, [11](#)

mon_analysis_port
 monitor, [26](#)

monitor, [24](#)
 build_phase, [25](#)
 mon_analysis_port, [26](#)
 new, [25](#)
 run_phase, [25](#)
 uvm_component_utils, [25](#)

new

- agent, 16
- coverage, 18
- driver, 21
- env, 23
- monitor, 25
- register_file_sequence, 30
- scoreboard, 32
- test, 36
- num_errors
 - scoreboard, 33
- num_packets
 - register_file_sequence, 30
- NUM_REGS
 - reference_model.c, 38
- o_rs1_ID
 - pkt, 28
 - register_file.v, 39
- o_rs2_ID
 - pkt, 28
 - register_file.v, 40
- parent
 - sequencer, 34
- pkt, 26
 - i_data_WB, 28
 - i_instr_ID, 28
 - i_rd_WB, 28
 - i_write_en_WB, 28
 - o_rs1_ID, 28
 - o_rs2_ID, 28
 - uvm_field_int, 27
 - uvm_object_utils_begin, 27
- reference_model Directory Reference, 13
- reference_model.c
 - NUM_REGS, 38
 - register_file_golden, 38
- reference_model/reference_model.c, 37
- register_file.v
 - always, 39
 - clk, 39
 - i_data_WB, 39
 - i_instr_ID, 39
 - i_rd_WB, 39
 - i_rst_ID, 39
 - i_write_en_WB, 39
 - o_rs1_ID, 39
 - o_rs2_ID, 40
 - registers, 40
- register_file_golden
 - reference_model.c, 38
 - scoreboard.sv, 51
- register_file_sequence, 29
 - body, 30
 - new, 30
 - num_packets, 30
 - uvm_object_utils, 30
- registers
 - register_file.v, 40
- req_pkt
 - driver, 21
- rst
 - tb_top.sv, 41
- rtl Directory Reference, 13
- rtl/register_file.v, 38
- rtl/register_file_wrapper.sv, 40
- run_phase
 - driver, 21
 - monitor, 25
 - test, 36
- run_test
 - tb_top.sv, 41
- s_interface, 30
 - Interfaces, 12
- scoreboard, 31
 - ap_imp, 33
 - build_phase, 32
 - check_phase, 32
 - new, 32
 - num_errors, 33
 - uvm_component_utils, 32
 - write, 32
- scoreboard.sv
 - register_file_golden, 51
- sequencer, 33
 - parent, 34
 - uvm_component_utils, 34
- tb Directory Reference, 14
- tb/s_interface.sv, 40
- tb/tb_top.sv, 40
- tb_top.sv
 - forever, 41
 - rst, 41
 - run_test, 41
 - U_REGISTER_FILE_WRAPPER, 41
 - vif, 41
- test, 35
 - build_phase, 36
 - end_of_elaboration_phase, 36
 - m_sequence, 36
 - new, 36
 - run_phase, 36
 - uvm_component_utils, 36
- Testbench components, 11
- U_REGISTER_FILE_WRAPPER
 - tb_top.sv, 41
- uvm Directory Reference, 14
- UVM verification components, 12
- uvm/agent.sv, 42
- uvm/coverage.sv, 43, 44
- uvm/driver.sv, 44, 45
- uvm/env.sv, 45, 46
- uvm/monitor.sv, 47
- uvm/pkt.sv, 48, 49

- uvm/register_file_pkg.sv, [49](#)
- uvm/register_file_sequence.sv, [50](#)
- uvm/scoreboard.sv, [51](#), [52](#)
- uvm/sequencer.sv, [53](#)
- uvm/test.sv, [54](#)
- uvm_component_utils
 - agent, [16](#)
 - coverage, [19](#)
 - driver, [21](#)
 - env, [23](#)
 - monitor, [25](#)
 - scoreboard, [32](#)
 - sequencer, [34](#)
 - test, [36](#)
- uvm_field_int
 - pkt, [27](#)
- uvm_object_utils
 - register_file_sequence, [30](#)
- uvm_object_utils_begin
 - pkt, [27](#)
- vif
 - tb_top.sv, [41](#)
- write
 - coverage, [19](#)
 - scoreboard, [32](#)